SC6113 / BLOCKCHAIN APPLICATIONS

GoalSave

A transparent, goal-based savings DApp on Ethereum Sepolia

SUBMISSION STATUS	Code and local tests complete. Replace the highlighted identity, URL, contract-address, and Sepolia screenshot fields before submission.

Student	Module	Submission
[STUDENT NAME] [STUDENT ID]	SC6113	Individual Assignment
GitHub	Render	Sepolia contract
[ADD URL]	[ADD URL]	[ADD ADDRESS]

PROJECT POSITIONING

GoalSave follows the taught Flask + MetaMask + Solidity workflow while implementing an original financial use case, real ETH accounting, authorization, event history, automated testing, and deployment-ready configuration.

Prepared: 18 October 2026 | Network: Sepolia (chain ID 11155111) | Educational use only

01 / COURSE BASELINE

What came from class - and what is original

The lecturer did not rely only on slides; the public examples and the progression of student repositories show a practical, copy-and-adapt teaching workflow. The correct baseline is therefore the stack and interaction pattern, not the submission completeness of one classroom repository.

Evidence from the lecturer and Congxiang repository

Evidence	Observed content	Interpretation
Lecturer SC6113 Gist (5 Aug 2026)	paynow contract stores payer, payee, and amount; transaction() reads them.	A first state-write/state-read exercise. It does not transfer ETH.
Congxiang - 16 Sep	Flask pages and Web3 transfer UI using weixin/check_transaction.	Strongly resembles a guided classroom increment.
Congxiang - 23 Sep	Deposit page using deposit_money/deposit_view.	Second guided interaction pattern.
Congxiang - 30 Sep / 7 Oct	Contract folder, then SQLite history added to a ChatGPT example.	A class exercise trail, not a polished final deliverable.

CONCLUSION	It is plausible that Congxiang's project follows the lecturer's live coding. However, it lacks a matched final contract/ABI set, systematic tests, deployment evidence, and a complete report, so submitting it unchanged would not safely satisfy the individual rubric.

How GoalSave stays within the taught scope

- Same approachable stack: one Solidity contract, MetaMask, Ethers.js, Flask, HTML, CSS, JavaScript, GitHub, and Render.
- Different business logic: personal savings targets instead of PayNow record storage, crowdfunding, or freelance escrow.
- One contract is appropriate for the individual assignment; group-project expectations such as three interacting contracts and multiple organizational roles are intentionally excluded.
- No source file from a classmate repository is included. The lecturer's public Gist is cited as a learning reference only.

02 / PROBLEM AND OBJECTIVES

A savings commitment users can verify

People often track savings goals in spreadsheets or banking notes. Those records are easy to change and do not give contributors a shared, independently verifiable history. GoalSave uses a smart contract to make the goal rules and transaction trail explicit.

Objectives

- Let a wallet create multiple named goals with a target amount and future deadline.
- Accept real Sepolia ETH deposits and update progress from on-chain state.
- Allow normal withdrawal only after reaching the target or passing the deadline.
- Provide an explicit emergency close path without hiding it in normal history.
- Show transaction feedback, goal status, and Etherscan links in a responsive interface.
- Validate input and handle wrong network, missing wallet, rejected transactions, and RPC failures.

Requirements coverage

Assignment requirement	GoalSave implementation	Evidence
Original financial DApp	Goal-based test-ETH savings with target/deadline rules	GoalSave.sol; UI
Solidity + ABI	Compiled 0.8.30 contract; generated standalone ABI	contracts/; abi/
MetaMask transactions	BrowserProvider signs create, deposit, and withdrawal calls	static/app.js
Flask backend	Pages, config/ABI API, validation, optional receipt lookup	app.py
HTML/CSS/JS	Responsive dashboard and event history	templates/; static/
Status/history	Events queried from deployment block; Etherscan deep links	5 event types
Validation/errors	Backend, browser, and contract checks	14 automated tests
Render readiness	Blueprint, Gunicorn, health endpoint	render.yaml; Procfile

SUCCESS CRITERION

A student can demonstrate create -> deposit -> locked withdrawal -> target reached -> successful withdrawal, with every successful state change visible both in the DApp and on Sepolia Etherscan.

03 / SYSTEM DESIGN

Wallet-first architecture with a thin backend

The blockchain is the source of truth. Flask serves the interface and performs supporting validation, while MetaMask keeps transaction signing under the user's control.

BROWSER	METAMASK	SEPOLIA
HTML/CSS dashboard Ethers.js contract client Event history	Account selection Network switch Transaction signature	GoalSave contract ETH custody Immutable event log
FLASK / RENDER	**JSON ENDPOINTS**	**OPTIONAL RPC**
Template/static hosting Gunicorn health check	ABI/config Goal/deposit validation	Receipt status lookup No private key stored

Primary transaction flow

- The page downloads the ABI and chain configuration from /api/config.
- The user connects MetaMask; the app verifies Sepolia chain ID 11155111.
- Flask validates form input before the browser constructs a contract call.
- MetaMask displays the exact transaction and the user signs it.
- After confirmation, the UI reloads contract state and queries events from the deployment block.
- The user can open the same transaction on Sepolia Etherscan for independent verification.

Why no database?

A database is optional in the individual brief. Goal ownership, saved amounts, status, and action history already exist on-chain. Avoiding a second writable data store prevents inconsistencies between SQLite and the contract. The backend remains meaningful through input validation, secure configuration delivery, health monitoring, and optional receipt verification.

04 / SMART CONTRACT

Rules enforced in Solidity

GoalSave.sol is a self-contained contract with no external imports. The compiled deployed bytecode is 4,040 bytes with the optimizer enabled and the Paris EVM target, which remains comfortably below the EIP-170 limit.

Function	Authorization / value	Effect
createGoal	Any wallet; no ETH	Validates name, target, and future deadline; assigns goal ID.
deposit	Any wallet; msg.value > 0	Adds ETH to one active goal; emits contributor and balance.
withdraw	Goal owner only	Unlocks after target or deadline; closes as Withdrawn.
emergencyWithdraw	Goal owner only	Returns current savings; closes as EmergencyClosed.
getGoal / getGoalIds	Read-only	Supports dashboard state without gas.

Security decisions

- Authorization: both withdrawal paths compare msg.sender with the stored owner.
- Checks-effects-interactions: savedAmount and status change before the external ETH call.
- Re-entrancy guard: a second withdrawal cannot enter while a payout is executing.
- Accounting integrity: direct receive/fallback transfers revert, so ETH cannot bypass a goal ID.
- Custom errors: smaller bytecode and clearer failure categories than repeated strings.
- Event separation: normal withdrawal and emergency close are distinguishable in the audit trail.

IMPORTANT DESIGN CHOICE

The contract uses msg.value as the deposited amount. This is safer than trusting a user-supplied amount parameter, which may not match the ETH actually sent.

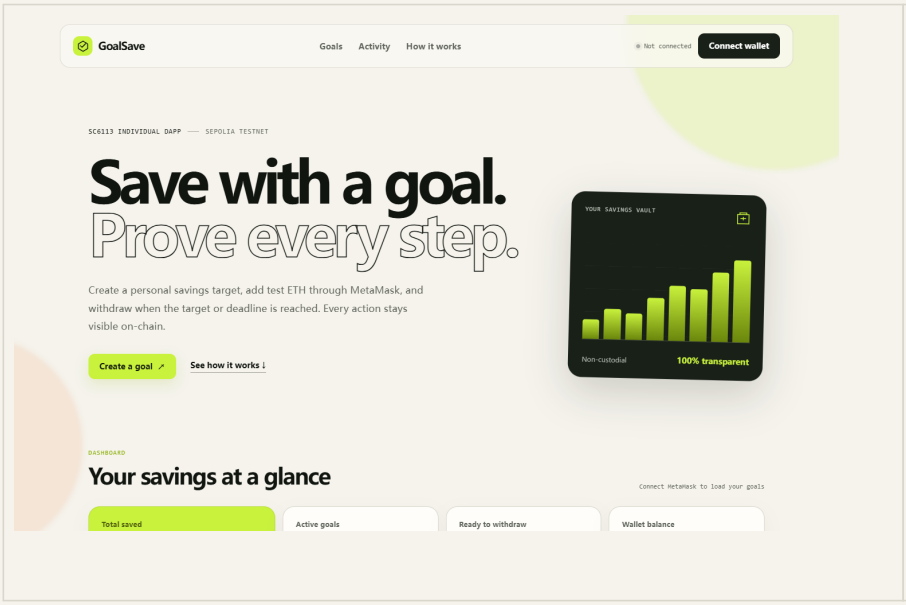
Goal lifecycle

State	Entry	Allowed next actions
Active	Goal created	Deposit; normal withdraw if unlocked; emergency close
Withdrawn	Normal withdrawal confirmed	Read-only; no further deposits or withdrawals
EmergencyClosed	Early close confirmed	Read-only; no further deposits or withdrawals

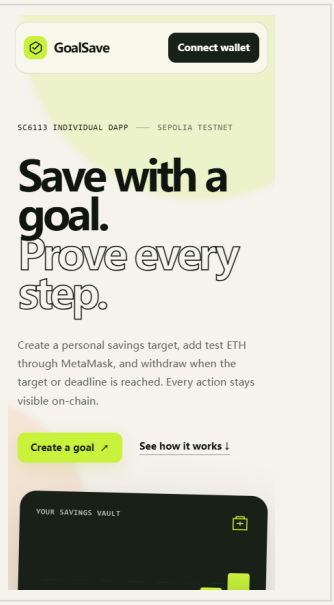
05 / APPLICATION IMPLEMENTATION

A complete DApp, not only a contract

The interface is designed as a small financial dashboard rather than a classroom form. It keeps wallet/network state visible and gives users clear pending, confirmed, and error feedback.



Desktop local preview



Mobile local preview

Implementation highlights

- Dashboard totals are calculated from contract state rather than hard-coded examples.
- Goal cards show saved amount, target, progress, deadline, status, and context-aware actions.
- Five event filters produce an on-chain activity table with Etherscan links.
- The JavaScript handles missing MetaMask, network switching, cancelled signatures, insufficient funds, and contract errors.
- Responsive breakpoints support laptop and mobile widths; reduced-motion preferences are respected.
- The Flask API normalizes decimals, limits names by UTF-8 bytes, validates deadlines, and rejects malformed transaction hashes.

SCREENSHOT STATUS	These are local interface previews, not deployment evidence. Replace or supplement them with screenshots showing the real Render URL, connected MetaMask account, confirmed goal, and Sepolia transaction before submission.

06 / TESTING

Fourteen automated checks passed locally

The contract was compiled with solc 0.8.30 and tested on an isolated in-memory Ganache chain. Flask routes and validation were tested with the Flask test client. No test needed a real private key or test ETH.

LOCAL RESULT	8 / 8 smart-contract tests passed | 6 / 6 Flask tests passed | JavaScript syntax check passed

ID	Automated scenario	Expected and observed result
C01	Create and index goal	Stored owner/name/target; owner ID list returned - PASS
C02	Third-party contribution	Goal savedAmount increased by msg.value - PASS
C03	Zero/missing-goal deposit	Both invalid calls reverted - PASS
C04	Early normal withdrawal	Reverted before target and deadline - PASS
C05	Target reached withdrawal	Funds released; state became Withdrawn - PASS
C06	Non-owner withdrawal	Authorization check reverted - PASS
C07	Emergency close	Funds returned; distinct terminal state recorded - PASS
C08	Direct ETH transfer	Receive/fallback rejected unattributed ETH - PASS
B01-B02	Home, health, and config	HTTP 200; ABI and Sepolia chain returned - PASS
B03-B05	Goal/deposit validation	Valid normalized; invalid values returned HTTP 400 - PASS
B06	Receipt endpoint	Hash validated; missing RPC handled gracefully - PASS

Required manual Sepolia checks

Check	Status before submission
Render URL opens and /health returns ok	TO BE COMPLETED BY STUDENT
MetaMask switches to Sepolia and connects	TO BE COMPLETED BY STUDENT
Create/deposit transactions confirmed	TO BE COMPLETED BY STUDENT
Invalid input and locked withdrawal demonstrated	TO BE COMPLETED BY STUDENT
Target reached and normal withdrawal confirmed	TO BE COMPLETED BY STUDENT
Etherscan receipts and post-refresh persistence captured	TO BE COMPLETED BY STUDENT

The full reproducible test plan and required screenshot filenames are provided in docs/TEST_PLAN.md.

07 / DEPLOYMENT AND EVALUATION

What remains account-dependent

The project includes every source-code and configuration item needed for submission. The final public links and chain evidence cannot be truthfully produced without the student's MetaMask, Sepolia test ETH, GitHub account, and Render account.

Final deployment sequence

Step	Action	Record in submission
1	Compile GoalSave.sol in Remix 0.8.30; deploy via MetaMask on Sepolia.	Address, tx hash, block, screenshot
2	Enter address/block in deployment.json; push project to own GitHub repository.	GitHub URL and commit
3	Create Render Blueprint; set CONTRACT_ADDRESS, DEPLOYMENT_BLOCK, CHAIN_ID.	Render URL and /health screenshot
4	Run the manual demonstration using small Sepolia ETH values.	Create/deposit/withdraw receipts
5	Replace all bracketed fields and TO BE COMPLETED rows in the final report.	Final 5-8 page PDF

Evaluation

GoalSave meets the individual assignment scope with a single original financial contract and a complete web stack. Its strongest features are correct ETH accounting, clear withdrawal authorization, event-driven history, three-layer validation, responsive transaction feedback, and automated negative testing. The principal limitation is that it is an educational testnet project, not an audited savings product. MetaMask RPC performance and faucet availability may also affect demonstrations.

Conclusion

The project advances beyond the lecturer's basic state-storage example without exceeding the course's practical stack. It is sufficiently small to explain during assessment, yet complete enough to demonstrate the full DApp lifecycle from Solidity and ABI through Flask, browser interaction, Sepolia evidence, GitHub, and Render.

FINAL CHECK

Do not submit this draft until name/ID, GitHub URL, Render URL, contract address, deployment transaction, and real Sepolia screenshots have been added and every placeholder removed.

References

[1] T. T. Teoh, '6113 AY2026 Aug - transferMoney.sol,' GitHub Gist, 5 Aug 2026. https://gist.github.com/teohteiktoe/84327003ab22a8829a681b84e4b3ad19

[2] Ethereum Foundation, 'Solidity documentation: Security considerations.' https://docs.soliditylang.org/

[3] MetaMask, 'Provider API.' https://docs.metamask.io/wallet/reference/provider-api/

[4] Render, 'Deploy a Flask app.' https://render.com/docs/deploy-flask

[5] SC6113 Individual Assignment brief supplied by the module teaching team.